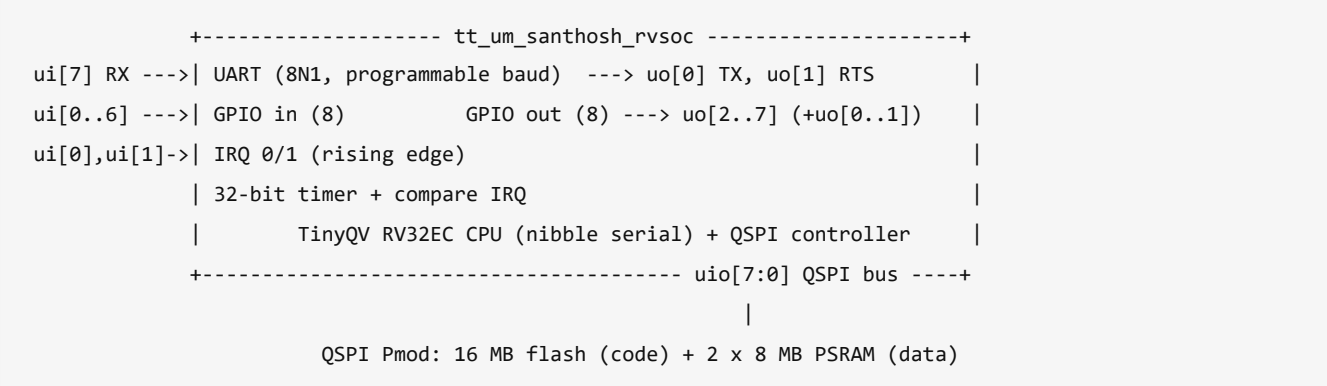
How it works

A 32-bit RISC-V microcontroller (RV32EC) in 2x2 Tiny Tapeout tiles. The CPU is **TinyQV** by Michael Bell (Tiny Tapeout 6, `tt_um_MichaelBell_tinyQV`, Apache-2.0, https://github.com/MichaelBell/tinyQV): it processes 4 bits per clock, executes code directly from an external QSPI flash and uses two external QSPI PSRAMs as RAM. There is no on-chip SRAM. This SoC adds a programmable-baud UART and a 32-bit timer with a compare interrupt, and removes the upstream SPI and debug outputs.



ISA: RV32E (16 registers) + C (compressed) + Zicsr, `ecall`, `ebreak`, `mret`. Implemented CSRs: `mstatus` (MIE, MPIE), `misa`, `mie`, `mip`, `mepc`, `mcause`. TinyQV specifics: addresses are 28 bits, code addresses 24 bits; `gp` (x3) is hard-wired to 0x1000400 and `tp` (x4) to 0x8000000; there is no misaligned-access or illegal-instruction trap. TinyQV's custom instructions (Zcb subset, multi-register load/store) are still present.

Changes from upstream TinyQV:

- `fence` executes as a NOP (upstream stalled).
- `csrrwi` / `csrrsi` / `csrrci` use their immediate (upstream used a register; immediate bit 4 reads as 0).
- Removed to fit 2x2: the custom 16-bit multiply (those encodings write 0) and the Zicntr counters (`rdcycle` / `rdtime` / `rdinstret` read 0; use TIMER_COUNT).

Vectors: reset 0x0000000, `ecall` / `ebreak` trap 0x0000004, interrupt 0x0000008.

Memory map

Address	Device
0x0000000 - 0x0FFFFFF	QSPI flash (16 MB, execute in place, read only)
0x1000000 - 0x17FFFFF	QSPI PSRAM A (8 MB)
0x1800000 - 0x1FFFFFF	QSPI PSRAM B (8 MB)
0x8000000 - 0x800003F	Peripheral registers (32-bit)

Peripheral registers

Address	Name	Access	Description
0x8000000	GPIO_OUT	R/W	Write: value for uo[7:0]. Read: current uo_out pins
0x8000004	GPIO_IN	R	ui_in[7:0]
0x800000C	GPIO_SEL	R/W	bit 0: uo[0] = GPIO_OUT[0] instead of UART TX; bit 1: uo[1] = GPIO_OUT[1] instead of UART RTS. Bits 7:2 read as 1 (uo[7:2] are always GPIO)
0x8000010	UART_DATA	R/W	Write: transmit a byte. Read: received byte (clears RX valid)

Address	Name	Access	Description
0x8000014	UART_STATUS	R	bit 0: TX busy, bit 1: RX byte valid
0x8000028	TIMER_COUNT	R/W	32-bit counter, +1 every 8 clocks (8 MHz at 64 MHz)
0x800002C	TIMER_CMP	R/W	Compare value. Timer IRQ is set when COUNT == CMP; writing CMP clears it
0x8000034	UART_DIV	R/W	Clocks per UART bit minus 1 (13 bits). Reset 555 = 115200 baud at 64 MHz

GPIO_OUT, GPIO_IN, GPIO_SEL, UART_DATA and UART_STATUS are at the same addresses as in the TT06 TinyQV.

Interrupts

`mcause` = 0x80000010 + n; enable with bit 16+n in `mie` (`mstatus.MIE` is 1 after reset).

n	Source	Type	Clear by
0	ui_in[0]	rising edge	clearing `mip` bit 16
1	ui_in[1]	rising edge	clearing `mip` bit 17
2	UART byte received	level	reading UART_DATA
3	Timer COUNT == CMP	level	writing TIMER_CMP

Code runs from QSPI flash and the stack lives in PSRAM, so a C interrupt handler that saves 10 registers takes about 2,400 clocks (37 us at 64 MHz). Keep timer periods well above that (the demo uses 100 ms). The UART receiver holds one byte: read it (or use interrupt 2) within one character time, about 87 us at 115200 baud.

Boot flow

1. While `rst_n` is low all `uio` pins are inputs. The QSPI read latency (0-5 clocks, normally 1-3) is read from SD2:SD0 (uio[4], uio[2], uio[1]).
2. The flash must already be in continuous-read (XIP) mode and the PSRAMs in QPI mode; the TT demo board's RP2040 sets this up before releasing reset.
3. After reset the CPU fetches its first instruction from flash address 0.

How to test

You need a QSPI Pmod (16 MB flash and two 8 MB PSRAMs) on the bidirectional Pmod header, with the demo firmware programmed at flash address 0.

1. Build the firmware: `make -C firmware` gives `firmware/build/rvsoc_fw.bin`.
2. Program `rvsoc_fw.bin` into the flash at offset 0 (TT demo board / QSPI Pmod tools).
3. Set the clock to 64 MHz (lower also works, but the baud rate scales with it).
4. Reset: drive `rst_n` high, then low; put the flash in continuous-read mode and the PSRAMs in QPI mode; drive all CS# high and SD2:SD0 = read latency (2 is a good start at 64 MHz); clock at least 8 times; release the QSPI lines, then raise `rst_n` while the clock is high.
5. Leave ui[2..6] low and connect a 3.3 V serial terminal at **115200 8N1** to uo[0] (TX) and ui[7] (RX). Expected output:

```
Hello, TT!
ISA PASS
PSRAM PASS
TIMER PASS
GPIO PASS
SELFTEST PASS
tick 10
tick 20
...
```

Then:

- **LEDs:** a single LED runs back and forth on uo[2]..uo[7], one step per 100 ms timer interrupt.
- **Echo:** characters you type are echoed back.
- **External interrupts:** a rising edge on ui[0] or ui[1] prints `IRQ0` or `IRQ1`.
- **GPIO mirror mode:** reset with ui[3:2] = 01 and uo[7:2] then follows ui[7:2].
- **ui[4] must be low:** it is a simulation-only fast mode (2 Mbaud, 250 us tick).

The full pin-by-pin guide is in `docs/IO_GUIDE.md`, and FPGA testing (PYNQ-Z2) is in `docs/fpga.md`.

External hardware

- QSPI Pmod with 16 MB flash and 2 x 8 MB PSRAM (https://github.com/mole99/qspi-pmod) on the bidirectional Pmod (uio).
- A 3.3 V USB-UART adapter or the TT demo board RP2040 UART on uo[0]/ui[7].
- Optional: buttons on ui[0]/ui[1] (interrupts) and LEDs on uo[2..7].

Credits

CPU core, QSPI controller and memory map: TinyQV by Michael Bell (Apache-2.0). UART: Ben Marshall (MIT), modified by Michael Bell. SoC integration, timer, programmable UART, firmware, tests and FPGA harness: Santhosh (2026).